Session 2.4

PyTorch, Curve Fitting, and Yield Forecasts

The training loop in code, a curve fitted, and a real yield series



© Zhigang Feng · Redistribution prohibited without permission

Zhigang Feng

What PyTorch Actually Is

tensors
arrays of numbers, as in NumPy

autograd
a record of every operation, so exact derivatives come back

nn.Module
a class that holds a network's parameters

optim
gradient-descent variants: SGD, Adam

- A scientific-computing library that became popular for deep learning, not the reverse.
- The first two pieces are the substance: write any scalar as a chain of array operations and its gradient is free.
- The last two are bookkeeping for parameters and update rules.
- **Everything in Deck 2.3 is four library calls. Today we make them.**

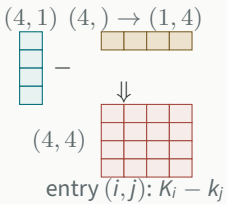
© Zhigang Feng Redistribution prohibited without permission

Source: A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library", *NeurIPS* 32, 2019.

Tensors: Shapes, Types, and the Broadcasting Bug

```
k = torch.linspace(0.5, 2.0, 4)
K = k.reshape(-1, 1)
print(k.shape, K.shape, K.dtype)
print((K - k).shape)
```

```
# shape (4,)
# shape (4, 1): four observations, one feature
# torch.Size([4]) torch.Size([4, 1]) torch.float32
# torch.Size([4, 4])   <- not what you meant
```

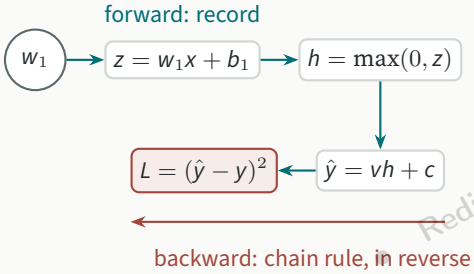


- A **tensor** is an array with a **shape** (4 rows, 1 column) and a **dtype** (32-bit floats).
- **Broadcasting** lines shapes up from the right, so a $(4,)$ is read as one row, $(1,4)$.
- Each size-1 axis is copied to match: the column across, the row down. Result: every K_i minus every k_j, silently.
- **Keep n observations of d variables as (n,d), and print the shape before you trust the number.**

Source: Session 2 lab notebook, block A1.

© Zhigang Feng
Reuse not prohibited without permission

Autograd: the Tape



- Mark a tensor `requires_grad=True` and PyTorch records every operation it enters: the **computation tape**.
- `backward()` walks the tape in reverse, one chain-rule step per operation: an exact derivative, not a finite difference.
- This is **autograd**: the chain rule, applied by the library.
- **You write the forward pass only.**

© Zhigang Feng
Redistribution prohibited without permission

`backward()`: One Step You Can Check by Hand

```
x, y = 2.0, 3.0
w1, b1, v, c = (torch.tensor(a, requires_grad=True) for a in (0.5, -0.1, 1.2, 0.3))
z = w1 * x + b1; h = torch.relu(z); yhat = v * h + c; L = (yhat - y) ** 2
L.backward()
# forward : z = 0.9000, h = 0.9000, yhat = 1.3800, L = 2.6244
# backward: dL/dv = -2.9160, dL/dc = -3.2400, dL/dw1 = -7.7760, dL/db1 = -3.8880
# update  : w1 -> 0.5778, v -> 1.2292;  loss 2.6244 -> 1.7488     (eta = 0.01)
```

- A four-parameter ReLU network on one observation. Each derivative is one chain-rule product, and the lab asserts every number against autograd.
- A second hidden unit with $z = -0.8 < 0$ receives a gradient of exactly 0.0: no signal, no learning.
- **If your hand derivative and autograd disagree, the hand derivative is wrong.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block A2; example from Miao & Feng, *AI for Economists*, §9.3.

The Training Loop Is Always the Same Five Lines

```
model = nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for step in range(300):
    prediction = model(X)                    # 1 forward
    loss = ((prediction - y) ** 2).mean()    # 2 loss
    optimizer.zero_grad()                    # 3 clear the old gradients
    loss.backward()                          # 4 backward
    optimizer.step()                         # 5 update
```

- Forward, loss, clear, backward, update. A language model with billions of parameters is trained by these five lines.
- Only line 1 (the model) and line 2 (the loss) change between applications.
- Fit a curve: *L* is the squared prediction error. Solve a model: *L* is the squared Euler residual, which is Deck 2.5.
- **Once derivatives are free, one algorithm covers both. Learn the loop once.**

Source: Session 2 lab notebook, block A3.

© Zhenlei Peng. Redistribution prohibited without permission

A Network Is a Class

```
class SimpleNet(nn.Module):
    def __init__(self, hidden=32, act=nn.ReLU):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(1, hidden), act(), nn.Linear(hidden, 1))
    def forward(self, x):
        return self.net(x)
```

- `nn.Module` is PyTorch's base class for anything with parameters. `__init__` declares the layers; `forward` says how an input flows through them.
- `nn.Linear(1, hidden)` is $Wx + b$ with W of shape hidden $\times$ 1; `nn.Sequential` composes layers in order.
- This is $f_\theta(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ from Deck 2.2, with 32 hidden units.
- **The class is the specification. Read it as you would read a regression equation.**

Source: Session 2 lab notebook, block A4.

© Zhigang Feng · Redistribution prohibited without permission

Optimizers, `zero_grad`, and Seeds

```
torch.manual_seed(1); net = SimpleNet()                # same seed, same initial draw
optimizer = torch.optim.Adam(net.parameters(), lr=3e-3, weight_decay=weight_decay)
optimizer.zero_grad()                                  # 3 clear the old gradients
loss = loss_fn(net(X_tr), y_tr); loss.backward(); optimizer.step()
net.eval()                                             # evaluation mode
with torch.no_grad():                                  # no tape: nothing to differentiate
    val = float(loss_fn(net(X_va), y_va))
```

- `parameters()` hands every W and b to the **optimizer**. `weight_decay` is an L2 penalty on the weights: ridge regression, in one argument.
- Gradients *accumulate* across calls to `backward()`: without `zero_grad()` you descend on the sum of every gradient so far.
- Initial weights are a random draw and a **seed** fixes it. `eval()` switches layers such as dropout; `no_grad()` turns the tape off when you only want predictions.
- **Forgetting `zero_grad()` raises no error. It trains the wrong model.**

Source: Session 2 lab notebook, blocks A4 and C.

© Zhigang Feng. Redistribution prohibited without permission

Start at Version 0, Where You Know the Answer

```
torch.manual_seed(0)
X = torch.randn(200, 2)
y = 1.0 + X @ torch.tensor([[2.0], [-0.5]]) + 0.1 * torch.randn(200, 1)
#  ... the five-line loop on nn.Linear(2, 1), 300 steps ...
# OLS    : [ 1.0073  1.9972 -0.5023]
# learned: [ 1.0073  1.9972 -0.5023]
assert np.allclose(ols, learned, atol=1e-3)
```

- A linear “network” trained on squared error must land on the OLS coefficients. It does, to four decimals.
- If it had not, the bug would be in the loop, and you would know before touching a real problem.
- The same habit in Deck 2.5: solve the growth model first where the policy is known in closed form.
- **Version 0 of any project is the case with a known answer.**

Source: Session 2 lab notebook, block A3.

© Zhiguo Feng Redistribution prohibited without permission

Session 2.4

A Curve Whose Answer Is Known

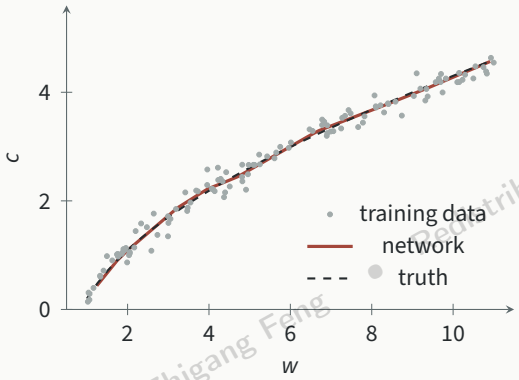
Fit it, then check it on data it has not seen

Redistribution prohibited without permission



© Zhigang Feng

Fit: a Network Finds $c = f(w)$



- A hidden law, $c = \ln w + 0.2\,w + \varepsilon$ on $w \in [1, 11]$, noise variance 0.0225; 300 training and 100 test observations.
- 32 ReLU units, 1,500 Adam steps. The functional form was never specified.
- Mean squared error: 0.0231 in training, 0.0229 on the test data.

Source: Session 2 lab notebook, block B.

© Zhigang Feng — Redistribution prohibited without permission

The Code, Block 1: Make the Data and Split It

```
torch.manual_seed(42)
w = torch.rand(400, 1) * 10 + 1
c = torch.log(w) + 0.2 * w + 0.15 * torch.randn(400, 1)
idx = torch.randperm(400); tr, te = idx[:300], idx[300:]
mu, sd = w[tr].mean(), w[tr].std()
```

- Line 2: 400 draws of *w*, uniform on $[1, 11]$, shape $(400, 1)$: *n* observations of one variable.
- Line 3: the hidden law plus noise with standard deviation 0.15, so the noise variance is $0.15^2 = 0.0225$.
- Line 4: shuffle the row numbers; the first 300 train, the last 100 test. A random split is fine for a cross-section. The yields, a time series, will be split by date.
- Line 5: the scaler's mean and standard deviation come from the training rows only.
- **We wrote the law ourselves, so we know the right answer before fitting anything.**

Source: Session 2 lab notebook, block B.

© Jugans Ref. Redistribution prohibited without permission

The Code, Block 2: the Network and the Training Loop

```
def fit_curve(hidden=32, act=nn.ReLU, epochs=1500, seed=0):
    torch.manual_seed(seed)
    net = SimpleNet(hidden, act)
    opt = torch.optim.Adam(net.parameters(), lr=1e-2)
    for _ in range(epochs):
        loss = ((net((w[tr] - mu) / sd) - c[tr]) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
    net.eval()
    return net
```

Redistribution prohibited without permission

- Lines 2–4: fix the initial draw; build `SimpleNet` (1 input, 32 ReLU units, 1 output); hand its Ws and bs to Adam.
- Line 6: forward on the standardised inputs, then the mean squared error against c.
- Line 7: clear, backward, update. The five-line loop, 1,500 times on all 300 rows.
- **Nowhere does the code say $\ln w$. The functional form is never written down.**

© Ehigiator Ferry

Source: Session 2 lab notebook, block B.

The Code, Block 3: Score It on Both Samples

```
net = fit_curve()
with torch.no_grad():
    mse = {name: ((net((w[i] - mu) / sd) - c[i]) ** 2).mean().item()
           for name, i in (("train", tr), ("test", te))}
print({k: round(v, 4) for k, v in mse.items()}, "| noise variance = 0.0225")
# {'train': 0.0231, 'test': 0.0229} | noise variance = 0.0225
```

- Line 2: `no_grad()` turns the tape off. We only want predictions now.
- Lines 3–4: the same formula on each sample, picked by its row numbers. The test rows are standardised with the training `mu` and `sd`.
- The noise variance, 0.0225, is the error even the true law would make. No model can beat it on average.
- Training and test errors are close to each other and to that floor.
- **The test number is the one to report: those 100 rows played no part in fitting.**

Source: Session 2 lab notebook, block B; the plotting lines are left out.

© Ehrane Fe... Redistribution prohibited without permission

Session 2.4

Next Month's 10-Year Yield

The data, the question, the split, and the fit



© Zhigang Feng
Redistribution prohibited without permission

The Data: Five FRED Series, Monthly

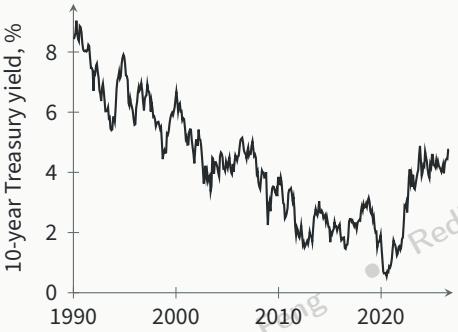
FRED series	What it is	How it enters
DGS10, DGS3MO	10-year and 3-month Treasury yields, daily	last trading day of each month
FEDFUNDS	effective federal funds rate, monthly	value for month $t-1$
UNRATE	unemployment rate, monthly	value for month $t-1$
CPIAUCSL	consumer price index, monthly	12-month inflation, month $t-1$

- Downloaded once, on 20 September 2026: 439 monthly rows, the last for August 2026.
- Yields are market prices, seen at month-end t. Macro series are published with a lag, so the value used at t is the one for month $t-1$.
- **Every input is something a forecaster could have known at the end of month t.**

Source: Federal Reserve Bank of St. Louis, FRED, fred.stlouisfed.org; Session 2 lab notebook, block C.

© Zhigang Feng Redistribution prohibited without permission

The Question

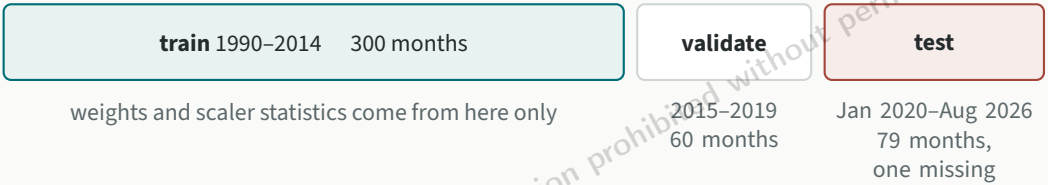


- At the end of month t, predict the 10-year yield at the end of month $t+1$.
- Inputs: the 10-year yield and the 10-year minus 3-month spread at t; funds rate, unemployment and inflation for $t-1$.
- The model: a small network that predicts the *change*; forecast = this month's level + change.
- **The aim is not to beat the bond market, but to fit a real series end to end.**

Source: FRED series DGS10, month-end, downloaded 20 Sep 2026.

© Zhigang Feng Redistribution prohibited without permission

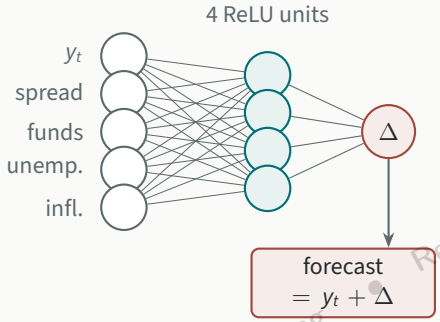
Splitting the Data by Date



© Zhiang Fang · Redistribution prohibited without permission

- Split by the *date of the label*, in order. No shuffle: next month is not a fresh draw.
- **Training** data fit the weights; **validation** data choose the width and when to stop; **test** data are used once, at the end.
- Inputs are standardised with the mean and standard deviation of 1990–2014 only.
- 79, not 80: the December 2025 label is missing, because FRED has no October 2025 CPI or unemployment value.
- **Anything estimated is estimated on the training window.**

The Network Behind the Forecast

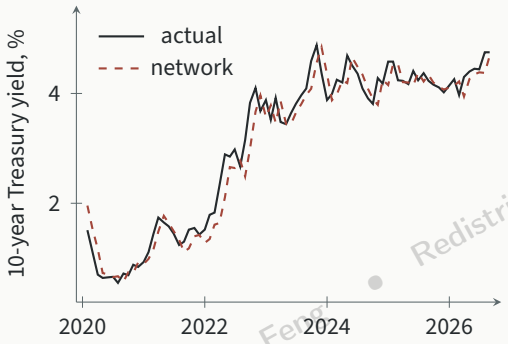


- Five standardised inputs, one hidden layer; the output is the *change*, so zero means “no change”.
- Width picked on validation from 4, 8, 16 (MSE 0.035, 0.047, 0.040): 4 units, 29 parameters.
- Full-batch Adam, step 0.003, weight decay 0.001 (ridge); stop after 40 epochs without a validation gain, keep the best.
- Ten seeds give test RMSEs of 0.27 to 0.36 points; the next slide averages them.
- **Small on purpose: 300 training months cannot pin down many parameters.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block C.

Actual and Network, Month by Month



- The 79 test months, never seen in training. Red: the forecast, averaged over ten seeds.
- It tracks the path through the 2020 low and the 2022 rise.
- Most of the tracking comes from starting at this month's level: the learned changes are small.
- **The training loop from earlier in this deck, unchanged, runs on a real economic series.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block C; FRED series DGS10, month-end, downloaded 20 Sep 2026.